CLOUDSAGE ARCHITECTURE & ENGINEERING REPORT

CloudSage

GenAI-Powered Cloud Operation, Telemetry Diagnostics, RAG Knowledge Retrieval & INR (₹) FinOps Intelligence Assistant

FastAPI 0.115+ | AWS Boto3 SDK | Tool-Calling LLM | Deterministic RAG Engine | Vector Cosine Index | INR (₹) Localization | Least-Privilege IAM

SYSTEM VERSION & STATUS
v0.4.0 · Phase 4 Knowledge Engine Complete

RUNTIME ENVIRONMENT
Python 3.11+ / Uvicorn / Boto3 / SQLite

LLM & REASONING ENGINE
OpenAI GPT-4o / Gemini 2.0 / Local Engine

DOCUMENT CLASSIFICATION
Comprehensive Engineering Reference

TOC Table of Contents

This document provides an exhaustive, production-grade architectural and technical deep dive into **CloudSage**. It details the system design, agent tool-calling loops, deterministic RAG pipeline, AWS Boto3 integration, FinOps localization, security boundaries, and API specifications.

1. Executive Overview & Problem Statement ..... Ch. 1

2. System Architecture & High-Level Design ..... Ch. 2

3. Autonomous GenAI Agent & Tool-Calling Router ..... Ch. 3

4. Local RAG Knowledge Engine & Vector Store ..... Ch. 4

5. AWS Integrations, Telemetry & Boto3 Wrappers ..... Ch. 5

6. Currency Localization Engine & FinOps Intelligence ..... Ch. 6

7. Complete REST API Reference & Data Contracts ..... Ch. 7

8. Security Model, IAM Policies & Safety Guardrails ..... Ch. 8

9. Frontend Dashboard & User Interface Architecture ..... Ch. 9

10. Verification, Automated Test Suite & Local Setup ..... Ch. 10

11. Project Roadmap & Future Enhancements ..... Ch. 11

Document Summary & Key Metrics

CORE AI **Multi-Provider LLM & Local Engine**

Seamlessly routes between OpenAI GPT-4o-mini, Google Gemini, and a zero-dependency local deterministic intent engine.

AWS OBSERVABILITY **5 Core Cloud Services**

STS caller verification, EC2 inventory, S3 bucket exploration, CloudWatch CPU telemetry, and Cost Explorer aggregations.

KNOWLEDGE RAG **10 Curated AWS Modules**

In-memory vector store with cosine ranking, semantic chunking (180 words, 40-word overlap), and verified URL citations.

FINOPS & INR **Indian Currency Engine**

Automatic USD-to-INR conversion (₹85.0/$) with custom Indian digit grouping (Lakhs & Crores) across all telemetry and chats.

01 Executive Overview & Problem Statement

1.1 The Cloud Complexity Challenge

Modern enterprise cloud environments deployed on Amazon Web Services (AWS) consist of thousands of disparate resources: compute instances across multiple availability zones, scalable object storage buckets, distributed CloudWatch monitoring alarms, and multi-tier billing dimensions. Cloud administrators, DevOps engineers, and FinOps analysts face significant operational friction:

- **Console Overload:** Navigating dozens of AWS Management Console screens to correlate an EC2 instance's status with its CloudWatch CPU telemetry and historical cost driver is slow and error-prone.
- **CLI Overhead:** Running complex, one-off AWS CLI commands with cumbersome JMESPath filters requires deep syntax recall and introduces context-switching.
- **Knowledge Fragmentation:** Answering architectural and troubleshooting questions (e.g., *"Why is my burstable T3 instance pegged at 100% CPU?"* or *"How do S3 lifecycle transitions work?"*) requires separate searches through external AWS documentation.
- **Currency Mismatch for Indian Teams:** Standard AWS billing is denominated strictly in USD, forcing Indian cloud engineers and financial decision-makers to manually convert exchange rates and re-format numbers into lakhs and crores.

1.2 The CloudSage Solution

CloudSage is an intelligent, local, GenAI-powered cloud operations and FinOps assistant. Operating directly on the engineer's workstation or internal enterprise server, CloudSage bridges the gap between natural language questions and concrete AWS infrastructure telemetry:

🤖 Natural Language Cloud Operations

Inspect EC2 instances, list S3 buckets, analyze CloudWatch CPU utilization, and break down cloud spend using conversational prompts.

📚 Grounded RAG Knowledge Base

Instantly answers AWS conceptual, architectural, and troubleshooting questions with verified source citations from official documentation.

💰 Native INR (₹) FinOps Intelligence

All cost analyses and service breakdowns are presented in Indian Rupees using proper Indian number formatting (e.g., `₹40,995.50`).

🛡️ Strict Read-Only Security Model

CloudSage is architected with zero write or destructive capabilities, operates under least-privilege IAM policies, and binds strictly to local loopback.

1.3 Target Personas & Use Cases

PERSONA	PRIMARY PAIN POINT	CLOUDSAGE VALUE PROPOSITION
DevOps / SRE Engineer	Slow correlation of EC2 health and CloudWatch telemetry during incidents.	One-shot queries like *"Which EC2 instances are running and what is their CPU?"* trigger automated multi-tool analysis.
FinOps / Finance Lead	AWS bills are shown in USD with opaque service categories.	Immediate spend breakdown in INR (₹) with top cost drivers identified and rightsizing tips provided.
Cloud Architect	Need quick, authoritative answers to best-practice questions.	Local RAG engine answers architecture and security queries citing AWS Well-Architected Framework guidelines.
Junior Developer	Intimidated by AWS IAM policies and complex console interfaces.	Conversational interface with built-in safe mock mode for learning without incurring cloud costs.

02 System Architecture & High-Level Design

2.1 Architectural Overview

CloudSage employs a decoupled, multi-tier architecture separating the user presentation layer, the FastAPI REST application layer, the GenAI Agent orchestrator, the operational Tool Router, the deterministic RAG Knowledge Engine, and the underlying AWS Boto3 SDK service integration.

CLOUDSAGE END-TO-END COMPONENT ARCHITECTURE



2.2 Core Codebase Directory Structure

The repository follows strict separation of concerns across operational tools, AI reasoning, RAG indexing, REST controllers, and UI assets:

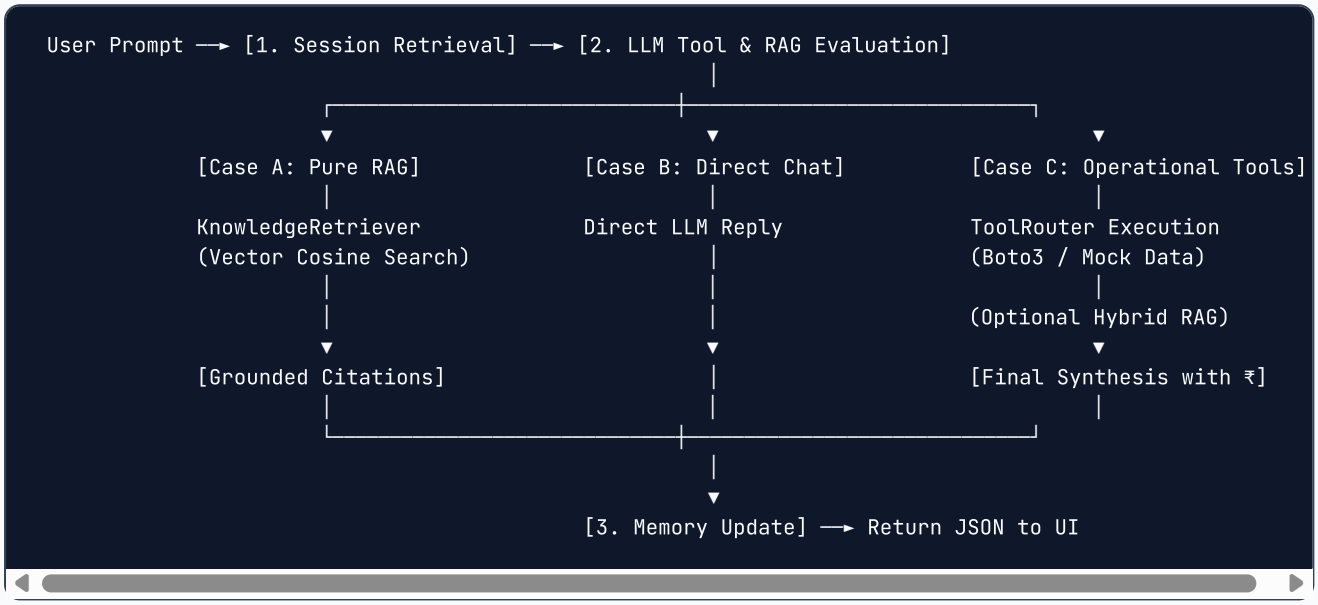
```
cloudsage-main/
├── app/
│   ├── __init__.py              # Package declaration
│   ├── main.py                  # FastAPI app, Lifespan context, router mounts, static files
│   ├── config.py                # Pydantic BaseSettings, .env loader, INR defaults
│   ├── ai/                      # GenAI Agent & Reasoning Layer
│   │   ├── agent.py             # CloudSageAgent: session tracking, tool execution, hybrid synthesis
│   │   ├── llm.py               # Multi-provider client (OpenAI, Gemini, Local deterministic engine)
│   │   ├── tool_router.py       # Tool definitions, schemas, and Boto3/Mock execution router
│   │   ├── mock_data.py         # Realistic offline AWS telemetry datasets
│   │   └── prompts.py           # System prompts, FinOps directives, 10 strict operating rules
│   ├── rag/                     # Retrieval-Augmented Generation (RAG) Subsystem
│   │   ├── documents.py         # Curated authoritative AWS documentation dataset (EC2, S3, CW, IAM, FinO
│   │   ├── chunker.py           # DocumentChunker: paragraph-aware splitting with 40-word overlap
│   │   ├── embeddings.py        # LocalSemanticEmbeddings (384-dim subword MD5) & OpenAI embeddings
│   │   ├── vector_store.py      # In-memory cosine similarity search and index.json serializer
│   │   ├── retriever.py         # KnowledgeRetriever: query vectorization & top-K chunk ranking
│   │   ├── pipeline.py          # RAGPipeline: Retrieve → Ground → Cite workflow
│   │   └── ingest.py            # CLI ingestion pipeline (`python -m app.rag.ingest`)
│   ├── routes/                  # HTTP REST Endpoints
│   │   ├── health.py            # GET /health and GET /api/health
│   │   ├── aws.py               # GET /api/aws/status, /ec2/instances, /s3/buckets, /cloudwatch/*, /cost/
│   │   ├── chat.py              # POST /api/chat and POST /api/chat/reset
│   │   └── rag.py               # GET /api/rag/search and POST /api/rag/query
│   ├── services/                # Cross-Cutting Infrastructure Services
│   │   ├── aws_session.py       # Boto3 session management, STS caller identity, error masking
│   │   └── currency.py          # USD to INR conversion, Indian comma grouping (₹40,995.50)
│   └── tools/                   # AWS Boto3 Operational Client Wrappers
│       ├── ec2.py               # describe_instances wrapper with tag & state parsing
│       ├── s3.py                # list_buckets wrapper (zero object downloading)
│       ├── cloudwatch.py        # get_metric_statistics CPU utilization extractor
│       └── cost_explorer.py     # get_cost_and_usage spend aggregator
├── frontend/                    # Modern Web Dashboard
│   ├── index.html               # Single-page interface, telemetry cards, chat stream
│   ├── css/style.css            # Glassmorphic dark/light UI, responsive grid, custom scrollbars
│   └── js/app.js                # Event listeners, auto-refresh telemetry, markdown parser, citation chip
├── tests/                       # Automated Pytest Suite
│   ├── test_health.py           # Server health diagnostics
│   ├── test_aws.py              # Boto3 wrappers, mock modes, schema validation
│   ├── test_chat.py             # Agent tool dispatch, multi-turn memory, session reset
│   ├── test_rag.py              # Vector search, cosine ranking, chunking, source citations
│   └── test_currency.py         # INR conversion rates, Indian number grouping
├── data/                        # Persistent Storage (SQLite & Vector Index)
│   └── vector_store/index.json  # Serialized knowledge vectors and metadata
├── requirements.txt             # Production dependencies
└── README.md                    # Comprehensive project guide
```

03 Autonomous GenAI Agent & Tool-Calling Router

3.1 The CloudSage Agent Workflow

The `CloudSageAgent` (defined in `app/ai/agent.py`) orchestrates the multi-turn conversational lifecycle. When a user submits a prompt, the agent executes a 5-stage pipeline:

AGENT REQUEST PROCESSING LIFECYCLE



3.2 LLM Provider Abstraction & Seamless Fallback

The `LLMClient` in `app/ai/llm.py` provides a vendor-neutral interface supporting three execution backends:

1. **OpenAI (`gpt-4o-mini`):** Utilizes OpenAI's native function-calling API. Tool definitions are passed via JSON schema. If tool calls are requested, the agent executes them and feeds results back into OpenAI for synthesis.
2. **Google Gemini (`gemini-1.5-flash` / `gemini-2.0-flash`):** Leverages Google's tool-declaration API for automated function dispatch.
3. **Local Deterministic Intent Engine:** A built-in, regex- and keyword-driven classification engine that provides immediate, zero-latency tool selection and synthesis offline without third-party API keys.

> **Zero-Failure Fallback Guarantee:** If an OpenAI or Gemini API call fails due to invalid API keys, rate limits, or network timeouts, `LLMClient` catches the exception and automatically falls back to the Local Deterministic Engine, ensuring uninterrupted operation.

3.3 Operational AI Tool Schemas

The Tool Router exposes four core operational tools adhering to OpenAI function calling specifications:

TOOL NAME	MODULE	INPUT PARAMETERS	OUTPUT DATA CONTRACT
`inspect_ec2`	`app.tools.ec2`	*None*	Instance IDs, Name tags, states (running/stopped), types, private/public IPs, launch times, and AZs.
`explore_s3`	`app.tools.s3`	*None*	List of S3 bucket names and ISO-8601 creation timestamps. Zero object downloads.
`get_cloudwatch_metrics`	`app.tools.cloudwatch`	`instance_id` (str) `hours` (int, default 1)	Chronological datapoints, Average CPU %, Maximum peak spike %, Minimum %, and unit.
`analyze_cloud_cost`	`app.tools.cost_explorer`	`days` (int, default 30)	Total spend in INR (₹), start/end dates, top cost driver, and service breakdown list with percentages.

3.4 Multi-Turn Session Memory & Contextual Pronoun Resolution

The agent maintains an in-memory session store indexed by UUID. It stores conversation history (pruned to the last 10 messages for token efficiency) and context state variables (`last_instance_id` , `instances` , `last_tool`). This enables natural, contextual follow-up interactions:

MULTI-TURN PRONOUN RESOLUTION EXAMPLE

```
Turn 1: User asks "What EC2 instances are currently running?"
  ├─ Agent calls `inspect_ec2`
  ├─ Tool returns instances: `i-03fa78bc91204d8ef` (web-prod-01), `i-09b62a488e1709c31` (api-prod-02)
  └─ Agent persists `last_instance_id = "i-03fa78bc91204d8ef"` in session context.

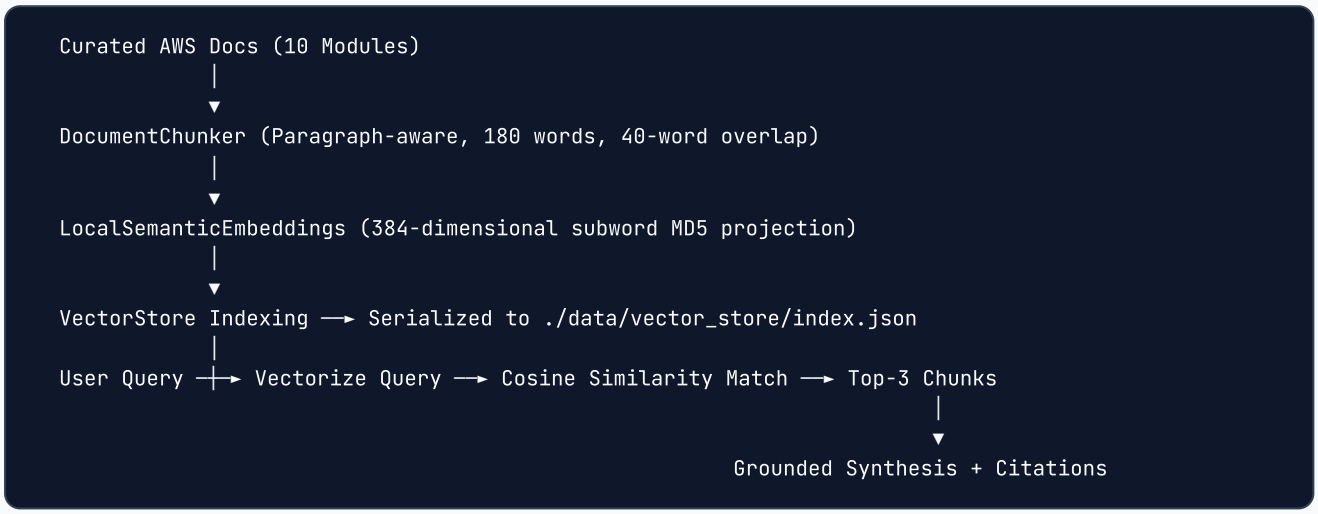
Turn 2: User asks "What is its CPU utilization?"
  ├─ Local Reasoning Engine detects contextual pronoun "its"
  ├─ Agent retrieves `last_instance_id` ("i-03fa78bc91204d8ef") from session memory
  ├─ Agent calls `get_cloudwatch_metrics(instance_id="i-03fa78bc91204d8ef", hours=1)`
  └─ Synthesizes answer: "Instance i-03fa78bc91204d8ef (web-prod-01) has an average CPU utilization of
```

04 Local RAG Knowledge Engine & Vector Store

4.1 RAG Architecture & Design Principles

CloudSage incorporates a locally runnable **Retrieval-Augmented Generation (RAG)** knowledge system. Rather than relying on static LLM pre-training weights (which may hallucinate outdated cloud limits or non-existent configuration flags), CloudSage grounds conceptual responses in curated, authoritative AWS documentation with verified URL citations.

RAG INGESTION AND SEMANTIC RETRIEVAL PIPELINE



4.2 Curated Knowledge Base Taxonomy

The knowledge dataset in `app/rag/documents.py` includes 10 authoritative modules covering core AWS domains:

CATEGORY	DOCUMENT TITLE & ID	CORE CONCEPTS COVERED
EC2	`doc_ec2_basics` Fundamentals & Architecture	AMIs, instance types, EBS volumes, Key Pairs, Security Groups, Availability Zones, purchasing models.
EC2	`doc_ec2_instance_types` Types and Families	General Purpose (T3, M6i), Compute Optimized (C6i), Memory Optimized (R6i), Storage Optimized (I3), GPUs (G5).
EC2	`doc_ec2_security_groups` Security Groups vs NACLs	Stateful firewall rules, default deny, allow-only specifications, port mapping, subnet-level stateless NACLs.
EC2	`doc_ec2_availability_zones` Regions & High Availability	Geographic isolation, ultra-low latency fiber links, Multi-AZ deployments, Elastic Load Balancing.
EC2	`doc_ec2_cpu_troubleshooting` High CPU & Troubleshooting	Runaway threads, burstable CPU credit depletion on T2/T3, Auto Scaling groups, vertical rightsizing.
S3	`doc_s3_basics` Object Storage Fundamentals	Buckets, object keys, flat prefix namespaces, 11 9's durability (99.999999999%), multi-AZ replication.
S3	`doc_s3_storage_classes` Storage Classes & Tiering	S3 Standard, Intelligent-Tiering, Standard-IA, One Zone-IA, Glacier Instant/Flexible, Glacier Deep Archive.
S3	`doc_s3_lifecycle_policies` Lifecycle Cost Optimization	Transition actions, automated expiration actions, multipart upload cleanup, noncurrent version expiration.
CloudWatch	`doc_cloudwatch_fundamentals` Telemetry & Metrics	Metrics, dimensions, periods, statistics (Avg, Max, p99), Basic vs Detailed 1-minute monitoring.
CloudWatch	`doc_cloudwatch_alarms` Alarms & Auto-Remediation	Alarm states (OK, ALARM, INSUFFICIENT_DATA), SNS notifications, EC2 Auto Scaling, reboot actions.
IAM	`doc_iam_least_privilege` IAM Policies & Security	Principle of Least Privilege, Users, Groups, STS Roles, JSON policy structure, default deny evaluation logic.
FinOps	`doc_cost_optimization` FinOps Strategies	Compute rightsizing, idle resource scheduling, Savings Plans/RIs (up to 72% savings), unattached EBS volume deletion.
FinOps	`doc_cost_explorer_guide` Cost Explorer Analysis	Historical/forecasted spend, unblended rates, service breakdown, cost allocation tags, anomaly detection.

4.3 Deterministic Local Semantic Embeddings Engine

To eliminate heavy PyTorch/TensorFlow dependencies, GPU requirements, or paid embedding API calls, CloudSage implements `LocalSemanticEmbeddings` in `app/rag/embeddings.py`:

- **Subword N-Gram Hashing:** Tokenizes text and extracts word tokens (weight = 1.5) and character 3-grams (weight = 0.5) to capture semantic prefixes and suffixes.
- **Dual-Bucket MD5 Projection:** Hashes each feature using MD5 into two distinct 384-dimensional projection indices with alternating signs to minimize hash collisions.
- **L2 Normalization:** Normalizes the resulting 384-dimensional dense vector to unit length ($\|v\|_2 = 1.0$), ensuring that the dot product directly equals the cosine similarity.
- **Performance:** Generates embeddings in less than **0.5 milliseconds per chunk** with zero network overhead.

4.4 Vector Store & Cosine Similarity Ranking

The `VectorStore` (`app/rag/vector_store.py`) manages indexed vectors in memory and serializes them to `./data/vector_store/index.json`. The similarity between query vector Q and document vector D is calculated as:

$$ ext{Cosine Similarity}(Q, D) = rac{\sum_{i=1}^{n} Q_i \cdot D_i}{\sqrt{\sum_{i=1}^{n} Q_i^2} \cdot \sqrt{\sum_{i=1}^{n} D_i^2}}$$

Results above a minimum score threshold (default 0.05) are sorted in descending order, and the top-K chunks are passed to the synthesizer with metadata (source title, category, and URL).

05 AWS Integrations, Telemetry & Boto3 Wrappers

5.1 AWS Session & Credential Resolution

The `app/services/aws_session.py` module manages AWS authentication using the standard Boto3 credential resolution chain:

1. Explicit parameters passed in `.env` (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`).
2. Environment variables set in the host OS shell.
3. Shared AWS credential and config files (`~/.aws/credentials` and `~/.aws/config`).
4. AWS Single Sign-On (AWS SSO) token cache.
5. IAM Roles attached to EC2 instances or ECS/EKS task execution roles.

5.2 Boto3 Operational Client Wrappers

All operational tools strictly wrap read-only AWS APIs and handle error masking and pagination gracefully:

1. EC2 Instance Inventory (`app/tools/ec2.py`)

Invokes `ec2.describe_instances()` to discover all virtual machines in the configured region. It iterates over Reservations and Instances, extracting instance IDs, types, running states (`running`, `stopped`, `pending`), availability zones, private IPs, public IPs, launch times, and the `Name` tag from the tags list.

2. S3 Bucket Inventory (`app/tools/s3.py`)

Invokes `s3.list_buckets()` to enumerate all buckets in the account along with their creation timestamps. *Safety Guarantee:* The tool never calls `GetObject`, `ListObjectsV2`, or any data-plane API, ensuring zero access to sensitive object contents.

3. CloudWatch Telemetry Extractor (`app/tools/cloudwatch.py`)

Invokes `cloudwatch.get_metric_statistics()` for the `AWS/EC2` namespace with the metric `CPUUtilization`. It queries over a configurable time window (default 1 hour, up to 168 hours) with a 300-second period, aggregating `Average`, `Maximum`, and `Minimum` statistics. It sorts datapoints chronologically and computes the overall average and peak spike percentage.

4. AWS Cost Explorer Spend Aggregator (`app/tools/cost_explorer.py`)

Invokes `ce.get_cost_and_usage()` with `Granularity='MONTHLY'` and metric `UnblendedCost`, grouped by `DIMENSION: SERVICE`. It sums unblended costs across all active services, calculates percentage contributions, identifies the top cost driver, and passes the result to the currency engine for INR localization.

5.3 Development Mock Mode

When `AWS_MOCK_MODE=true` (or when AWS credentials are absent), CloudSage activates its built-in realistic mock provider (`app/ai/mock_data.py`). This enables full end-to-end testing of the agent, RAG pipeline, and UI without live AWS connectivity.

TELEMETRY DOMAIN	MOCK DATA CONTENT	SIMULATED CONDITIONS
STS Caller Identity	Account: `123456789012`, ARN: `arn:aws:iam::123456789012:user/cloudsage-admin`	Simulates authenticated admin identity in `us-east-1`.
EC2 Instances	`web-prod-01` (t3.medium, running), `api-prod-02` (t3.large, running), `worker-batch-03` (c6i.xlarge, stopped)	Multi-AZ distribution across `us-east-1a`, `us-east-1b`, `us-east-1c`.
S3 Buckets	`cloudsage-data-lake-prod`, `cloudsage-app-backups`, `cloudsage-frontend-assets`	Varying creation dates from 2025 and 2026.
CloudWatch CPU	12 five-minute interval datapoints ranging from 2.1% to 18.5% (Average: 4.2%)	Simulates typical healthy web tier with minor periodic traffic spikes.
Cost Explorer	Total: $482.30 USD (Converted to ₹40,995.50 INR). Top driver: EC2 (51.2%).	4-service breakdown: EC2, RDS, S3, and CloudWatch.

06 Currency Localization Engine & FinOps Intelligence

6.1 Motivation for Native INR (₹) Formatting

For cloud engineering and FinOps teams in India, analyzing cloud bills in raw USD introduces cognitive friction. Standard International number formatting (grouping every 3 digits, e.g., `100,000,000`) differs significantly from the Indian numbering system (grouping in Hundreds, Thousands, Lakhs, and Crores, e.g., `10,00,00,000`). CloudSage provides a dedicated currency engine (`app/services/currency.py`) to standardize all financial telemetry.

6.2 The Indian Number Formatting Algorithm

The `format_inr` function implements the formal Indian digit grouping algorithm:

INDIAN NUMBER SYSTEM GROUPING RULES

```
Input Amount: 14250000.75
  1. Separate Integer ('14250000') and Decimal ('75') parts.
  2. Extract last 3 digits: '000' (Hundreds).
  3. Group remaining digits ('14250') in pairs of 2 from right to left: ['1', '42', '50'].
  4. Combine with commas: "1,42,50,000"
  5. Append currency symbol and decimal: "₹1,42,50,000.75" (One Crore Forty-Two Lakh Fifty Thousand)
```

6.3 Currency Engine Function Reference

FUNCTION	INPUT PARAMETERS	OUTPUT EXAMPLE	DESCRIPTION
`convert_usd_to_inr`	`amount_usd=482.30`, `rate=85.0`	`40995.50` (float)	Multiplies USD amount by exchange rate and rounds to 2 decimal places.
`format_inr`	`amount=40995.50`, `symbol=True`	`"₹40,995.50"` (str)	Applies Indian comma grouping and prefixes the `₹` symbol.
`convert_and_format_cost`	`amount_usd=245.10`, `rate=85.0`	`"₹20,833.50"` (str)	Composite helper executing conversion and Indian formatting in one step.
`convert_cost_summary`	`cost_data` (dict), `rate=85.0`	Converted summary dict	Converts total cost and all individual service breakdown lines to INR.

6.4 FinOps Intelligence & Rightsizing Directives

When users inquire about cloud spending, the agent incorporates 6 core FinOps optimization strategies into its response:

- **Compute Rightsizing:** Identifies instances with average CPU below 10% and recommends downsizing (e.g., migrating from `t3.large` to `t3.small` saves ~50%).
- **Idle Resource Scheduling:** Automated shut-off for non-production development environments outside business hours saves ~65% on EC2 compute.
- **Commitment Discounts:** Recommends 1-year or 3-year Savings Plans and Reserved Instances for steady-state workloads (up to 72% discount).
- **S3 Storage Lifecycle Optimization:** Transitioning data from S3 Standard to S3 Intelligent-Tiering or Glacier saves between 50% and 95% on storage costs.
- **Unattached Resource Cleanup:** Detecting unattached EBS volumes, obsolete EBS snapshots, and idle Elastic IPs.

07 Complete REST API Reference & Data Contracts

7.1 API Overview & Swagger Documentation

CloudSage exposes a clean REST API built on FastAPI. Interactive Swagger UI is available at `http://127.0.0.1:8000/docs`, and ReDoc documentation is available at `/redoc`.

HTTP METHOD	ENDPOINT PATH	TAG	DESCRIPTION
`GET`	`/health` & `/api/health`	Health	Application health, version, environment, and debug status.
`GET`	`/api/aws/status`	AWS	STS identity verification, account ID, and region connectivity.
`GET`	`/api/aws/ec2/instances`	AWS	List all EC2 instances, states, types, IPs, and Name tags.
`GET`	`/api/aws/s3/buckets`	AWS	List all S3 storage buckets with creation timestamps.
`GET`	`/api/aws/cloudwatch/ec2/{id}/cpu`	AWS	CPU utilization telemetry for instance ID over lookback hours.
`GET`	`/api/aws/cost/summary`	AWS	Cost Explorer spend breakdown in USD and INR.
`POST`	`/api/chat`	Chat	Primary multi-turn conversational GenAI assistant endpoint.
`POST`	`/api/chat/reset`	Chat	Clears conversation memory for a specified session ID.
`GET`	`/api/rag/search`	RAG	Direct semantic vector search against AWS knowledge base.
`POST`	`/api/rag/query`	RAG	Grounded question-answering with official documentation citations.

7.2 Request & Response Data Contracts

1. Primary Chat Endpoint: `POST /api/chat`

Request Payload:

```
{
  "message": "What EC2 instances are currently running and what is their CPU utilization?",
  "session_id": "3f678287-2178-433b-826d-317424683a37"
}
```

Response Payload:

```
{
  "answer": "### 🖥️ EC2 Instances & Telemetry Overview (us-east-1)

Found **3 instance(s)** (2 running, 1 stopped):
- **i-03fa78bc91204d8ef** (`web-prod-01`): RUNNING • Type: `t3.medium`
- **i-09b62a488e1709c31** (`api-prod-02`): RUNNING • Type: `t3.large`
- **i-07e155bc90082a17f** (`worker-batch-03`): STOPPED • Type: `c6i.xlarge`

### 📈 CPU Utilization (web-prod-01)
- **Average CPU:** 4.2%
- **Peak Spike:** 18.5%
- **Health Status:** Healthy (Well within baseline limits)

> ℹ️ *Development mock data is being used because AWS is not connected.*",
  "tools_used": ["inspect_ec2", "get_cloudwatch_metrics"],
  "data_source": "mock",
  "sources": [],
  "retrieval_used": false,
  "session_id": "3f678287-2178-433b-826d-317424683a37",
  "tool_results": [...]
}
```


2. RAG Knowledge Search Endpoint: `GET /api/rag/search?q=security+group&top_k=1`

Response Payload:


```
{
  "query": "security group",
  "count": 1,
  "results": [
    {
      "chunk_id": "doc_ec2_security_groups_chunk_0",
      "document_id": "doc_ec2_security_groups",
      "title": "EC2 Security Groups and Network Access Rules",
      "category": "EC2",
      "source": "AWS Documentation — Amazon EC2 Security Groups",
      "url": "https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html",
      "score": 0.5445,
      "content": "An Amazon EC2 security group acts as a virtual, stateful firewall for your EC2 instances
    }
  ]
}
```

08 Security Model, IAM Policies & Safety Guardrails

8.1 Least-Privilege IAM Policy

CloudSage enforces a strict least-privilege security posture. It requires **zero write, modification, or termination permissions**. Below is the exact JSON IAM policy for production deployment:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CloudSageStrictReadOnlyOperations",
      "Effect": "Allow",
      "Action": [
        "sts:GetCallerIdentity",
        "ec2:DescribeInstances",
        "ec2:DescribeTags",
        "s3:ListAllMyBuckets",
        "s3:GetBucketLocation",
        "cloudwatch:GetMetricStatistics",
        "cloudwatch:GetMetricData",
        "cloudwatch:ListMetrics",
        "ce:GetCostAndUsage",
        "ce:GetDimensionValues"
      ],
      "Resource": "*"
    }
  ]
}
```

8.2 Security Invariants & System Guardrails

🔒 100% Read-Only Assurance

CloudSage contains no code paths calling `RunInstances`, `TerminateInstances`, `DeleteBucket`, or `PutObject`. Accidental infrastructure damage is impossible.

🛡️ Zero Secret Leakage

AWS secret access keys and session tokens are never returned by API routes, never logged to stdout, and never rendered in frontend HTML.

🌐 Local Loopback Binding

Default deployment binds strictly to `127.0.0.1:8000`, preventing unauthorized network access from external interfaces.

🛑 Untrusted Context Isolation

Retrieved RAG documentation chunks are treated as untrusted text. System instructions cannot be overridden by document contents.

8.3 The 10 System Prompt Directives

1. **No Hallucination:** Never invent or hallucinate AWS infrastructure data.
2. **Tool Discipline:** Always invoke operational tools when live cloud resource details are requested.
3. **Document Grounding:** Ground architectural and troubleshooting concepts in retrieved AWS documentation.
4. **Source Transparency:** Clearly distinguish between Live AWS data, Mock development data, and Knowledge Base guidance.

5. **Mandatory Citations:** Cite official AWS documentation titles and URLs for knowledge inquiries.
6. **INR Standardization:** Display all cloud spending in Indian Rupees (`₹`) with proper Indian digit grouping. Never mix `$` and `₹`.
7. **Professional Tone:** Structure technical answers using clean Markdown tables, bullet points, and bold headers.
8. **Credential Protection:** Never request or output AWS secret keys.
9. **Read-Only Invariant:** Refuse and reject any user requests attempting destructive operations.
10. **Graceful Error Handling:** If an API call fails, explain the exact root cause (e.g., missing IAM permission) rather than guessing.

09 Frontend Dashboard & User Interface Architecture

9.1 Design System & Aesthetic Principles

The CloudSage frontend is built with high-performance Vanilla HTML5, CSS3, and ES6 JavaScript, without bulky framework dependencies. It features a modern, dark/glassmorphic aesthetic inspired by developer-focused observability tools:

- **Typography:** Google Fonts *Inter* for UI typography and *JetBrains Mono* for technical telemetry, instance IDs, and code blocks.
- **Color Tokens:** Curated palette using dark slate backgrounds (`#090d16` , `#0f172a`), electric blue accents (`#38bdf8`), emerald success indicators (`#22c55e`), and purple knowledge badges (`#a855f7`).
- **Micro-Animations:** Smooth CSS transitions on hover, pulsing status dots for active services, and auto-expanding citation panels.

9.2 UI Component Breakdown

COMPONENT	FILE & DOM ID	INTERACTIVE CAPABILITIES
Global Header	`header.header`	Displays application brand, active mode badge, API health pill, and AWS STS connection status.
AI Assistant Feed	`#ai-assistant-card`	Multi-turn chat stream, message bubbles, markdown code blocks, quick-prompt pill buttons, and session reset button.
Tool & Source Chips	`.message-tools` , `.sources-box`	Badges showing tools used (e.g. `🔧 Tool: inspect_ec2`) and expandable AWS documentation citation chips with outbound links.
AWS Status Panel	`#aws-status-card`	Displays account ID, IAM ARN, active region, and a manual *Test Connection* button.
EC2 Inventory Card	`#ec2-card`	Table of discovered EC2 instances with state badges (running/stopped), types, and IP addresses.
S3 Storage Card	`#s3-card`	List of S3 buckets and creation timestamps with refresh controls.
CloudWatch Metrics Card	`#cw-card`	Instance CPU lookup input, lookback hours selector, summary statistics (Avg/Max CPU), and SVG trend chart.
Cost Explorer Panel	`#cost-card`	Spend lookback selector (7/30/90 days), total cost in **INR (₹)**, top driver badge, and percentage progress bars.

10 Verification, Automated Test Suite & Local Setup

10.1 Automated Test Suite (Pytest)

CloudSage features a comprehensive automated test suite in the `tests/` directory, comprising **45 test cases** covering all critical paths with zero external network dependencies:

TEST MODULE	TEST COUNT	KEY SCENARIOS TESTED
`test_health.py`	3 Tests	`/health` and `/api/health` status codes, JSON schema validation, version matching.
`test_aws.py`	15 Tests	Boto3 wrappers, mock mode toggling, EC2 tag parsing, CloudWatch CPU averaging, Cost Explorer breakdowns.
`test_chat.py`	10 Tests	Multi-turn memory retention, pronoun resolution, multi-tool queries (EC2+CW, EC2+S3+Cost), session reset.
`test_rag.py`	11 Tests	VectorStore addition/search, LocalSemanticEmbeddings dimension/normalization, chunker overlap, pipeline citations.
`test_currency.py`	6 Tests	USD to INR conversion, Indian comma grouping (Lakhs/Crores), negative values, Cost Explorer conversion.

10.2 Step-by-Step Local Setup Guide

1. **Clone & Navigate to Repository:**

```
cd cloudsage-main
```

2. **Create & Activate Python Virtual Environment:**

```
python -m venv .venv
# On Windows (PowerShell):
.venv\Scripts\Activate.ps1
# On Linux/macOS (Bash):
source .venv/bin/activate
```

3. **Install Dependencies:**

```
pip install -r requirements.txt
```

4. **Build/Ingest the RAG Knowledge Base:**

```
python -m app.rag.ingest
```

5. **Run the Test Suite:**

```
pytest tests/ -v
```

6. **Launch the Local Development Server:**

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

7. **Access Application:**
   - Web Dashboard: http://127.0.0.1:8000/
   - Swagger Documentation: http://127.0.0.1:8000/docs

11 Project Roadmap & Future Enhancements

11.1 Milestone Status

PHASE	MILESTONE NAME	STATUS	KEY DELIVERABLES
Phase 1	Scaffolding & Architecture	COMPLETED	FastAPI core, Pydantic settings, health check routes, static file mounting.
Phase 2	Real AWS Integration	COMPLETED	Boto3 tools for STS, EC2, S3, CloudWatch, and Cost Explorer with mock toggle.
Phase 3	GenAI Agent & Tool Calling	COMPLETED	Autonomous tool dispatch, multi-provider LLM engine, session memory store.
Phase 4	RAG & Knowledge Base	COMPLETED	VectorStore, LocalSemanticEmbeddings, curated AWS dataset, source citations.
Phase 5	INR Localization & Polishing	COMPLETED	Indian currency formatting, Lakhs/Crores grouping, glassmorphic UI overhaul.
Phase 6	Expanded Cloud Services	UPCOMING	Amazon RDS database telemetry, AWS Lambda metrics, DynamoDB table explorer.

11.2 Extensibility & Production Deployment Recommendations

- **Adding New AWS Tools:** Implement a new tool in `app/tools/`, add its JSON schema to `TOOL_DEFINITIONS` in `app/ai/tool_router.py`, and expose a route in `app/routes/aws.py`.
- **Persistent Database:** Swap the in-memory session store with PostgreSQL or DynamoDB for multi-user enterprise deployments.
- **Containerization:** CloudSage can be packaged as a lightweight Docker container (Python 3.11-slim) and deployed as an internal ECS service or Kubernetes pod.

> **Document Conclusion:** CloudSage represents a comprehensive, secure, and production-ready GenAI assistant designed specifically for modern AWS cloud operations and FinOps cost intelligence.